

VSCode插件开发七：玩转侧边栏自定义视图

上一篇已经讲了如何创建树视图，但如果你不想在原来的默认侧边栏面板上做修改，而是想自己定义一个侧边栏，因为你的内容可能不属于这些面板上相关的内容，比如包管理视图。下面就来详细讨论如何实现这个目标。

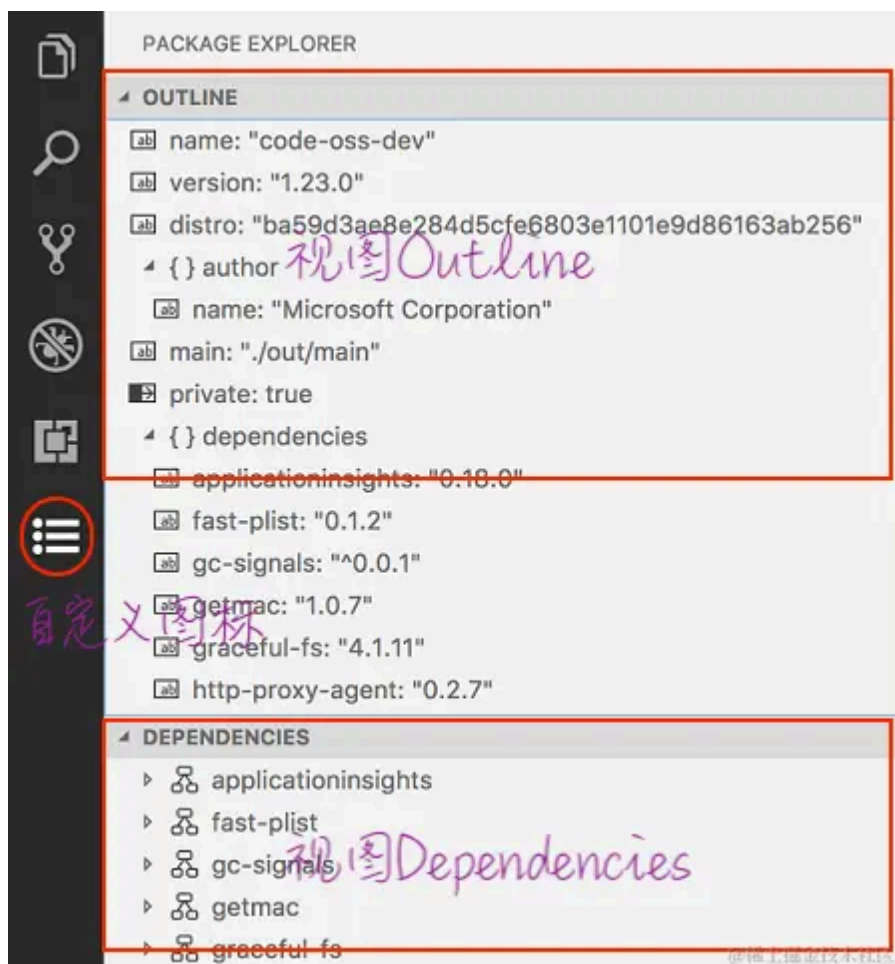
package.json 配置

首先，还是从 package.json 配置开始。首先，你需要定义一个在活动栏上的视图容器，然后再定义视图列表。如下所示：

```
{
  "contributes": {
    "viewsContainers": {
      "activitybar": [
        {
          "id": "package-explorer",
          "title": "Package Explorer",
          "icon": "resources/package-explorer.svg"
        }
      ]
    },
    "views": {
      "package-explorer": [
        {
          "id": "package-dependencies",
          "name": "Dependencies"
        },
        {
          "id": "package-outline",
          "name": "Outline"
        }
      ]
    }
  }
}
```

```
}  
}
```

在上面的配置中，我们在活动栏（`activitybar`）上添加了一个图标、标题和 id。视图列表（`views`）的定义方式你应该在上一篇文章中了解了，这里定义了视图的 id 和名称，其中 `package-explorer` 与活动栏的 id 完全一致。配置完成后，效果如下：



当然，你还需要在视图面板中添加内容，这与前一篇文章中定义树视图的方法完全相同。

webview 视图

`package.json` 类型配置

在定义树视图后，你可能会发现树视图的功能有一些限制，无法用于创建复杂的页面。因此，你可能会考虑使用Webview视图，这也是完全可行的。首先，你需要在 `package.json` 中声明视图类型，如下所示：

```
"package-explorer": [  
  {
```

```

        "type": "webview",
        "id": "package-dependencies",
        "name": "Dependencies"
    },
    {
        "id": "package-outline",
        "name": "Outline"
    }
]

```

在上面的配置中，我在第一个视图（**Dependencies**）中添加了 `"type": "webview"`，这样就可以在代码中使用Webview视图了。

视图提供器

与之前一样，你需要编写一个视图提供器类。基本结构如下：

```

import * as vscode from 'vscode'

class DependenciesProvider implements vscode.WebviewViewProvider {
    context: vscode.ExtensionContext
    constructor(context: vscode.ExtensionContext) {
        this.context = context
    }
    // 实现 resolveWebviewView 方法，用于处理 WebviewView 的创建和设置
    resolveWebviewView(webviewView: vscode.WebviewView, context: vscode.WebviewViewResolveCont
        // 配置 WebviewView 的选项
        webviewView.webview.options = {
            enableScripts: true,
            localResourceRoots: [this.context.extensionUri]
        };

        // 设置 WebviewView 的 HTML 内容，可以在这里指定要加载的网页内容
        webviewView.webview.html = "HTML 内容"
    }
}

```

同样，你还需要注册这个类：

```

vscode.window.registerWebviewViewProvider('package-dependencies', new DependenciesProvider(c
// 或者
vscode.window.createTreeView('package-dependencies', {

```

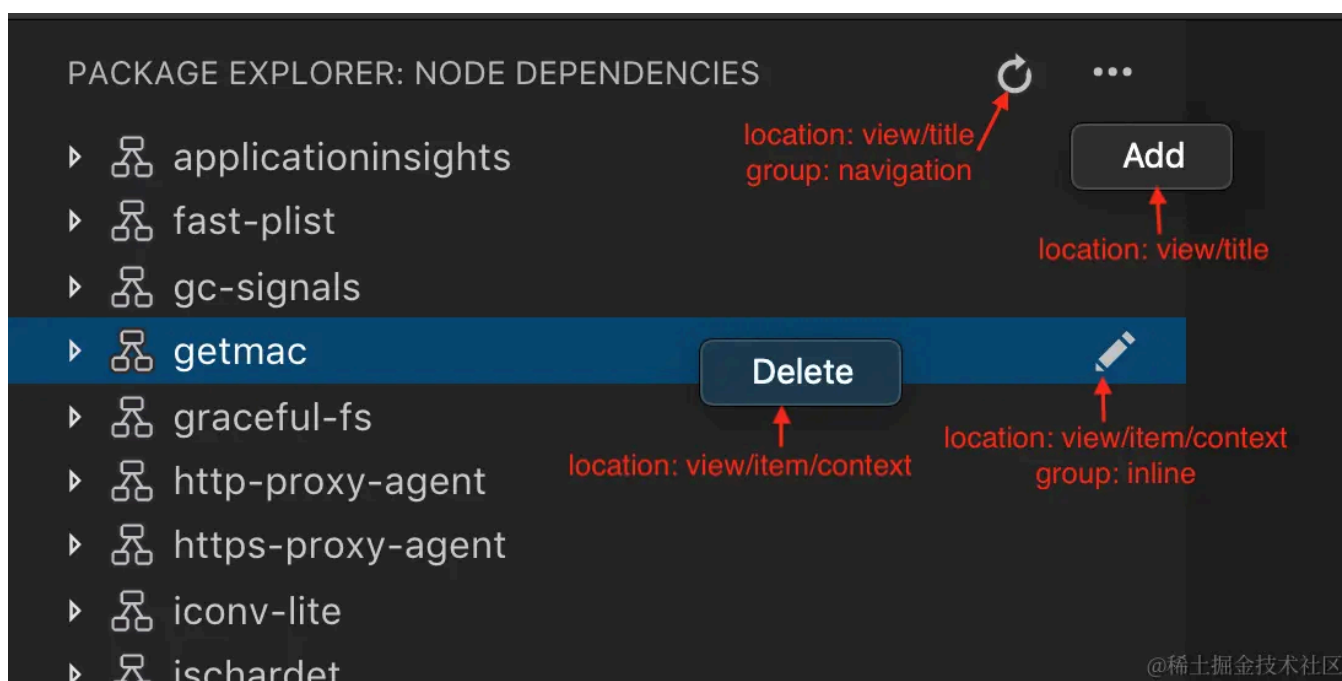
```
treeDataProvider: new DependenciesProvider(context),
})
```

这样，当你点击活动栏图标时，就可以看到自定义侧边栏的界面了。

更多功能

操作按钮

为了提供更丰富的用户交互体验，通常需要在视图添加各种操作（Actions），例如刷新、添加、编辑、删除等。这些操作可以通过菜单和视图标题栏等位置提供给用户。



我们可以看到图片上有4个位置的操作按钮。

1. 添加和刷新按钮

添加和刷新按钮位于标题栏上，需要使用view/title位置声明按钮。刷新按钮不需要点击“...”区域展开菜单，而“Add”按钮需要。以下是配置示例：

```
"contributes": {
  "commands": [
    {
      "command": "package-outline.refreshEntry",
      "title": "Refresh",
      "icon": {
```

```

        "light": "resources/light/refresh.svg",
        "dark": "resources/dark/refresh.svg"
    }
},
{
    "command": "package-outline.addEntry",
    "title": "Add"
}
],
"menus": {
    "view/title": [
        {
            "command": "package-outline.refreshEntry",
            "when": "view == package-outline",
            "group": "navigation"
        },
        {
            "command": "package-outline.addEntry",
            "when": "view == package-outline"
        }
    ]
}
}

```

可以看到，两者的不同之处在于刷新按钮添加了 `"group": "navigation"`。而 `"when": "view == package-outline"` 用于定义哪个视图应用这些操作，这里表示Outline视图。

2. 删除和编辑按钮

删除和编辑按钮针对单个项，需要在 `view/item/context` 下定义。其中，删除按钮是右键菜单中的选项，而编辑图标仅在鼠标悬停在该项上时显示。以下是它们的配置示例：

```

"contributes": {
    "commands": [
        {
            "command": "package-outline.editEntry",
            "title": "Edit",
            "icon": {
                "light": "resources/light/edit.svg",
                "dark": "resources/dark/edit.svg"
            }
        },
        {
            "command": "package-outline.deleteEntry",
            "title": "Delete"
        }
    ]
}

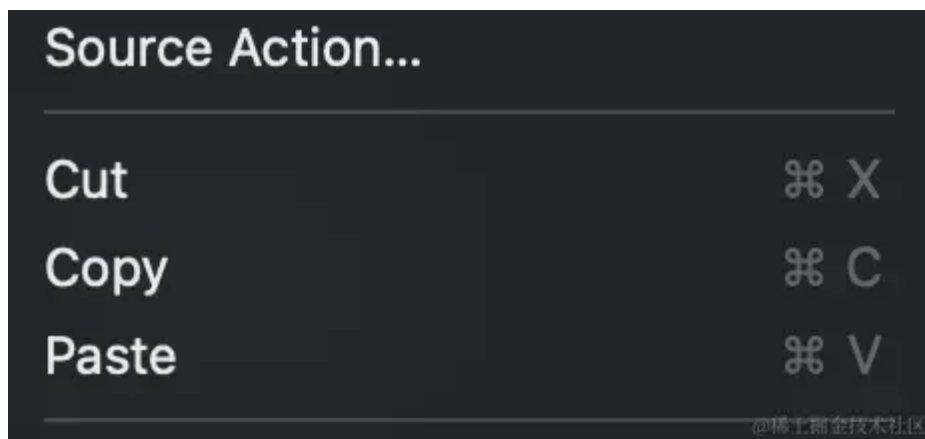
```

```

],
"menus": {
  "view/item/context": [
    {
      "command": "package-outline.editEntry",
      "when": "view == package-outline",
      "group": "inline"
    },
    {
      "command": "package-outline.deleteEntry",
      "when": "view == package-outline"
    }
  ]
}
}
}

```

这两者之间的不同之处同样在于 `group` 的不同。`group` 中的 `navigation` 和 `inline` 是默认的，你也可以随意命名，随意命名的话会出现在“Add”或“Delete”按钮的位置，并且不同的 `group` 会进行分组显示。如同下图所示：



默认情况下，按钮会按字母顺序排列。如果你想按自己的顺序排列，可以在 `group` 名称的末尾添加 `@[数字]`。

欢迎内容

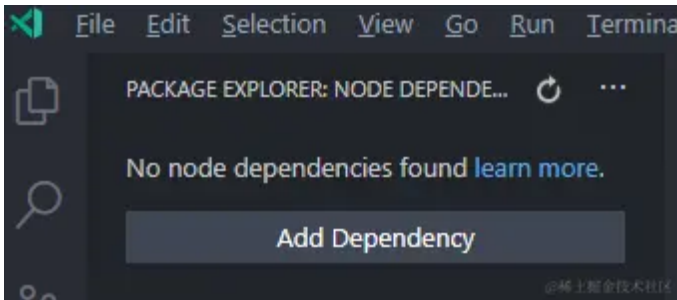
这个应该很常见，例如，在新建一个VSCode窗口并且还没有打开任何项目时，项目视图会显示文本提示和添加按钮。这个配置可以在 `viewsWelcome` 内进行配置，内容（`contents`）可以使用Markdown格式编写。

```

"contributes": {
  "viewsWelcome": [
    {

```

```
    "view": "package-outline",  
    "contents": "No node packages found [learn more](https://www.npmjs.com/).\n[Add Packag  
  }  
]  
}
```



通过上述文章，你应该已经更深入地了解了如何自定义侧边栏视图。